CI/CD Pipeline for a Calculator REST API

CSE 816 Software Production Engineering · Mini-Project Report

Mayank Tanwar (IMT2023012) · mayanktanwar01234@gmail.com · 30 October 2026

Code: github.com/Mayanktan/calculator-devops · Image: hub.docker.com/r/mayanktan/calculator-devops

1. Overview

A Spring Boot 3.3 REST calculator (`add`, `subtract`, `multiply`, `divide`, `power` under `/api/v1`, plus `/actuator/health`) with a fully automated pipeline: every push to `main` on GitHub is built, tested, packaged into a Docker image, pushed to Docker Hub, deployed with Docker Compose behind an Nginx reverse proxy, and the result is emailed. Divide by zero and missing or non-numeric parameters return HTTP 400 with a JSON error body. **Tests:** 8 JUnit 5 unit tests cover every operation including divide by zero, and 8 Spring MockMvc integration tests call the real endpoints; `mvn test` writes the JUnit XML that Jenkins publishes. **Tools:** Java 21, Maven, Git and GitHub, Jenkins LTS 2.528.2, Docker Engine 29.5 in Colima, Docker Compose, Nginx 1.27 and Gmail SMTP, all on one MacBook (Apple Silicon, 8 GB).

2. Architecture

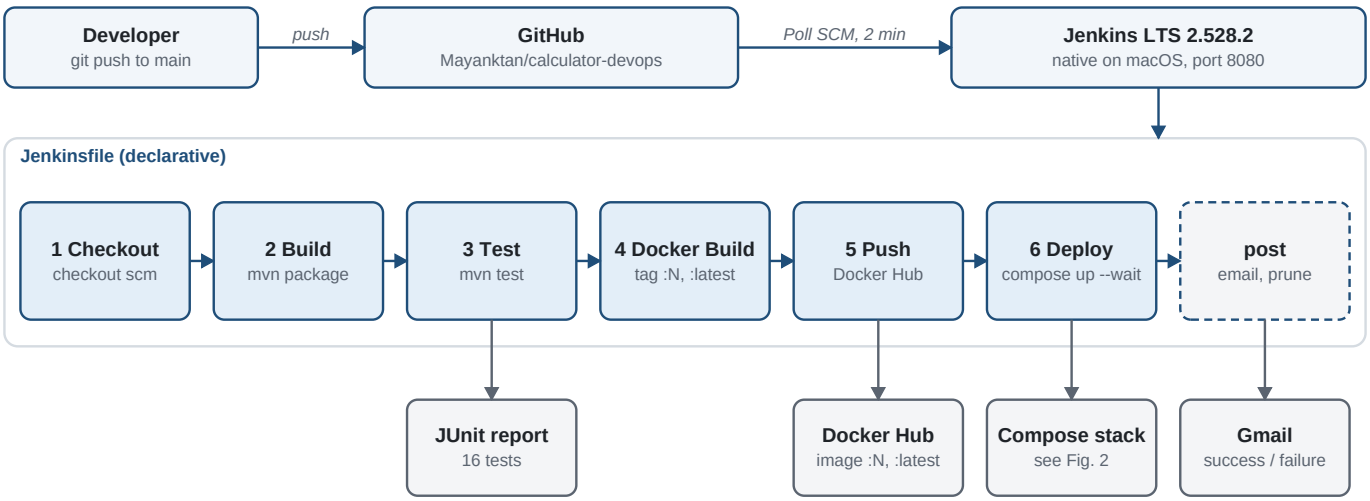


Fig. 1 Pipeline: Jenkins picks up each push and runs six stages; the side outputs are the test report, the pushed image, the deployed stack and the email.



Fig. 2 Deployment: only Nginx is reachable from the host; the app's port 8080 stays inside `calc-net`.

3. Pipeline stages

Stage	What it does	Fails the build when
1. Checkout	`checkout scm` from `main` (the implicit checkout is off, so there is exactly one)	GitHub is unreachable
2. Build	`mvn -B -ntp clean package -DskipTests` builds the executable JAR	the code does not compile
3. Test	`mvn test`; `junit` publishes `target/surefire-reports/*.xml`, even when tests fail	any test fails
4. Docker Build	`docker build` tagged `mayanktan/calculator-devops:$BUILD_NUMBER` and `:latest`	the image cannot be built
5. Push	`withCredentials(dockerhub-creds)`: `docker login --password-stdin`, push both tags, logout	login or push is rejected
6. Deploy	`docker compose up -d --remove-orphans --wait`, then `/actuator/health` through Nginx (up to 12 tries, 5 s apart)	the app is not healthy within 1 min
post	HTML email on success and on failure; prunes old dangling images	never

- **Git and GitHub:** public repository with a README (build, test, run, API calls); a `.gitignore` written first (excludes `target/`, IDE files and secrets); conventional-commit history; 5 feature branches merged through 5 pull requests; no credentials committed.
- **Trigger and secrets:** Poll SCM every 2 minutes, which the spec allows; Jenkins listens only on localhost without a login, so it is not exposed for a webhook. The Docker Hub token and the Gmail App Password live in the Jenkins credentials store; the

Jenkinsfile only names their IDs.

- **Idempotent re-runs:** a unique image tag per build, `compose up -d` recreates only what changed, a fixed Compose project name, and `disableConcurrentBuilds()`.
- **Containers:** two-stage Dockerfile (Maven + JDK 21, then JRE 21 alpine with only the JAR), dependency layer cached, non-root user `app`, `EXPOSE 8080`, `.dockerignore`; 331 MB (about 91 MB compressed). A single `docker run -p 8080:8080` answers on the mapped port. Compose adds network `calc-net`, the health check, `restart: unless-stopped`, environment variables and Nginx as the second component.

4. Results

Build	Started by	Result	Time	Tests	What it showed
#1	first run (command line)	SUCCESS	180 s	16/16	all six stages; first upload of every image layer
#2	Build Now (web page)	SUCCESS	37 s	16/16	re-run over a live stack, cached layers
#3	SCM change: landing page	SUCCESS	40 s	16/16	started 77 s after the push; new page live
#4	SCM change: broken test	FAILURE	17 s	15/16	stopped at Test; later stages skipped
#5	SCM change: revert	SUCCESS	40 s	16/16	back to green
#6	SCM change: reports commit	SUCCESS	147 s	16/16	fresh clone of the repository; deployed image 6

Build #4 failed **on purpose**: one assertion was changed to expect 2 + 3 = 6 to prove that a failing test stops the pipeline. Docker Build, Push and Deploy were skipped, no tag 4 was pushed, the live app stayed on build #3, and the failure email arrived; build #5 reverted the change. Every build emailed its result (Figs. 6 and 7).

5. Challenges and fixes

Problem	Cause	Fix
Health check through Nginx returned **HTTP 400**, so every Deploy would fail	The `/actuator/` route sent no Host header, so Nginx used the upstream name `calc_backend`; Tomcat rejects the underscore	Upstream renamed to `calc-backend`; proxy headers set for every route
Only 8 of 16 tests ran	Surefire runs only `*Test.java` by default, so `CalculatorControllerIT` was skipped	Explicit Surefire includes for `*Test.java` and `*IT.java`
Nginx kept old config after a redeploy	Compose replaced only the app; Nginx reads its config at start	`depends_on: restart: true` restarts Nginx after each app update
`release version 21 not supported` in Jenkins	The Mac's default JDK is 17, and launchd gave Jenkins no Homebrew PATH	Global Jenkins variables `JAVA_HOME` (JDK 21) and `PATH+EXTRA`
Docker broken after Docker Desktop was removed	`~/.docker` still used the Desktop credential helper and dead plugin links	Backed up and cleaned `~/.docker`; installed Colima with Compose and Buildx

Future work (capstone): Ansible provisioning, Kubernetes using the Actuator probes already enabled, Prometheus and Grafana, a secured Jenkins with a GitHub webhook, and a multi-architecture image.

6. Screenshots

Figs. 3 and 6-9 are the screenshots the problem statement requires: green pipeline run, email notification, docker ps and sample API responses. Terminal figures show real command output.

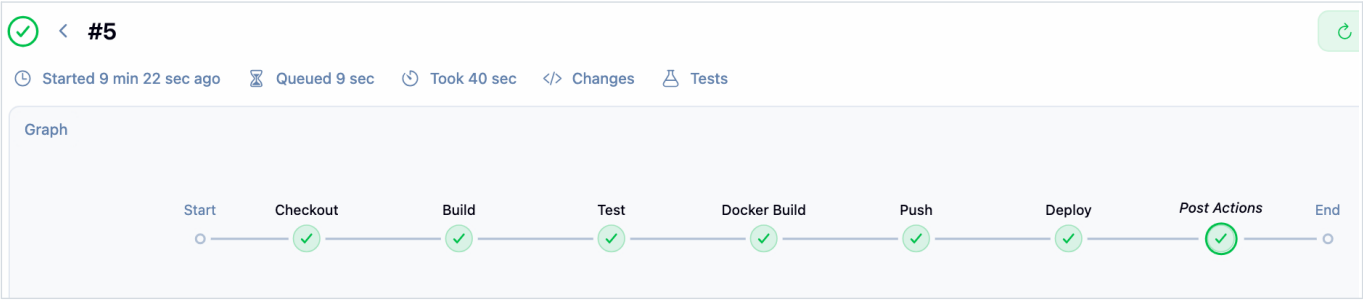


Fig. 3 Required: green pipeline run. Build #5, all six stages passed, then the email step.

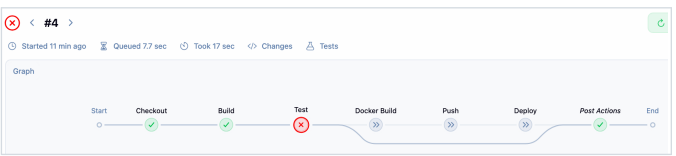


Fig. 4 Build #4: the Test stage failed (on purpose), so Docker Build, Push and Deploy were skipped.

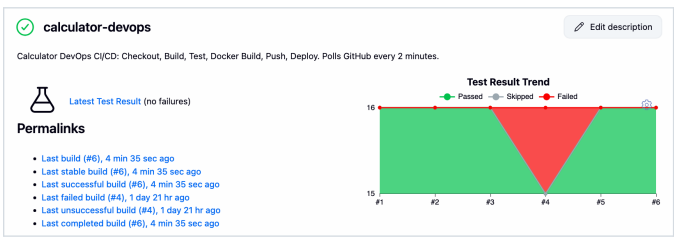


Fig. 5 Jenkins job: test result trend for builds #1-#6.

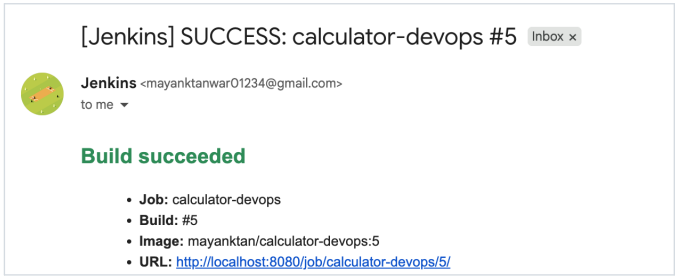


Fig. 6 Required: email notification, success (build #5).

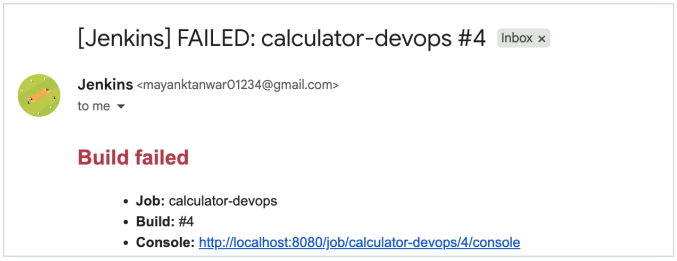


Fig. 7 Required: email notification, failure (build #4).

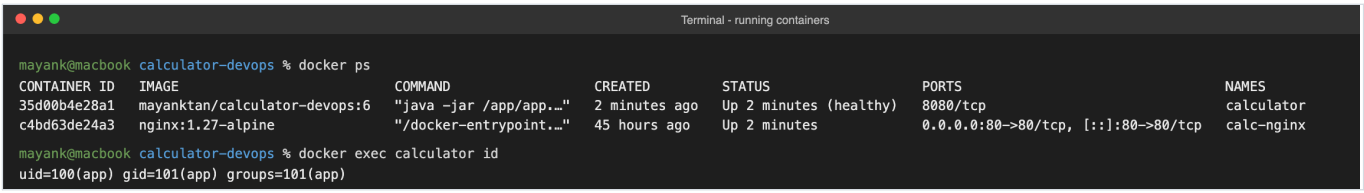


Fig. 8 Required: docker ps showing the running containers: calculator healthy on internal port 8080, Nginx published on port 80; the app runs as non-root uid 100.



Fig. 9 Required: sample API responses through Nginx, including HTTP 400 JSON errors.

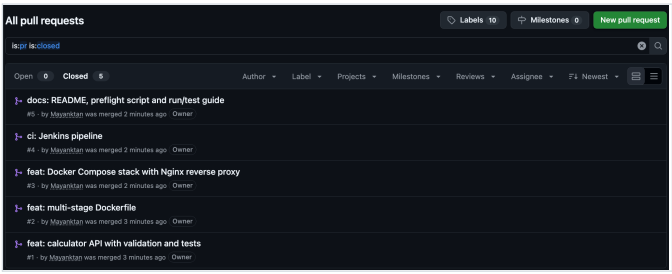


Fig. 10 GitHub: the five merged pull requests.

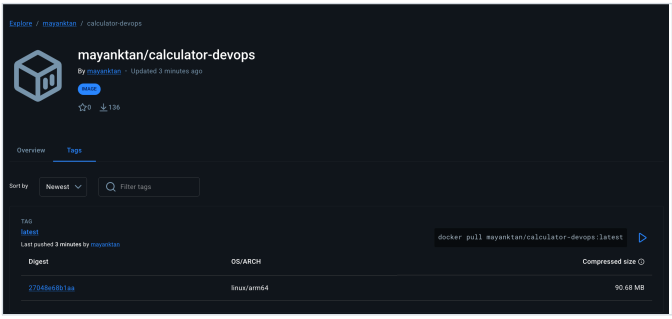


Fig. 11 Docker Hub: public image, one tag per build plus `latest`.

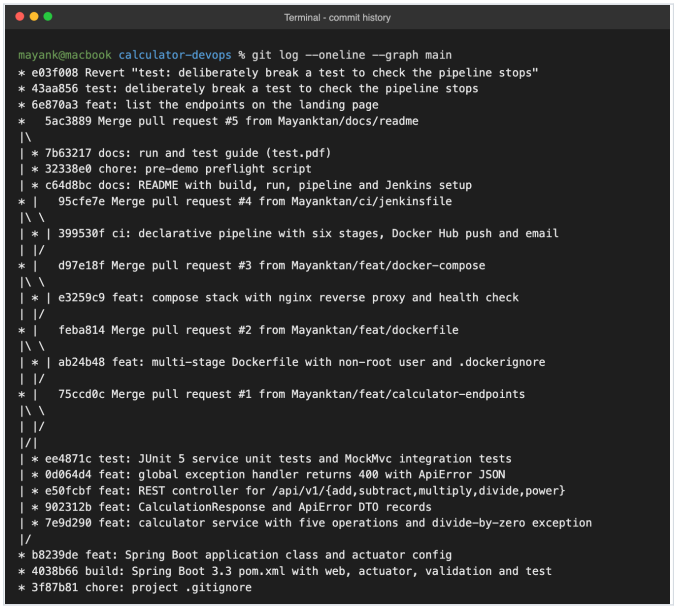


Fig. 12 Commit history of `main`: feature branches merged through pull requests, then the pipeline tests.